

0.21.2. Descendre avec `deplace_y` — doublé, deux positions

Le A descend en colonne 0 (y), puis le B en colonne 10 (yb).

1. Ce qu'il faut comprendre

C'est le 0.21 et le 0.21.1 réunis : deux `deplace_y`, chacun avec sa variable (`y`, `yb`).

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1619 octets** de cartouche.

1. UNE VARIABLE — PAS

```
uint8_t pas = 5;
```

2. UNE VARIABLE — Y

```
uint8_t y = 0;           // le A
```

3. UNE VARIABLE — YB

```
uint8_t yb = 0;          // le B
```

4. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
    y = deplace_y(0, y, ALPHABET[0], pas);    // le A, colonne 0
    yb = deplace_y(10, yb, ALPHABET[1], pas); // puis le B, colonne 10

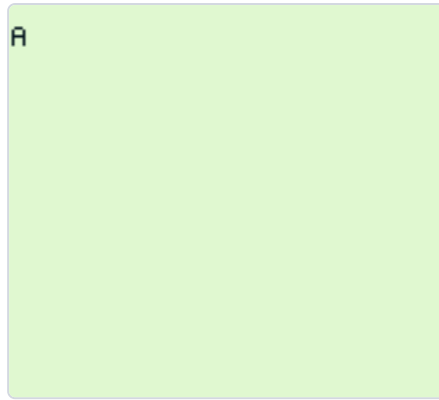
    while (true) {
        image();
    }
}
```

3. Ce qui se passe, étape par étape

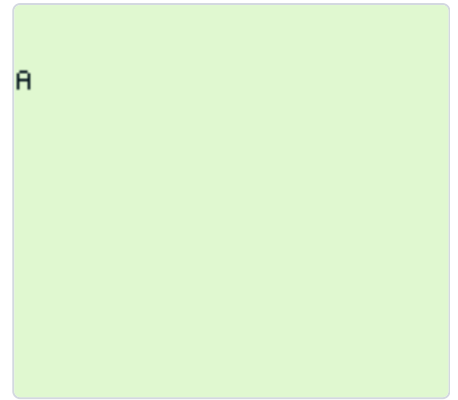
La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



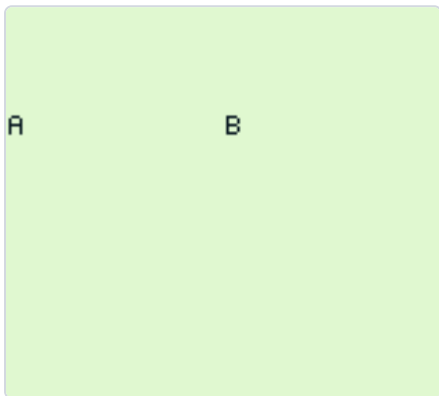
la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20



au bout d'une seconde — image 60



au bout de trois secondes — image 180

4. Ce qu'on doit voir

Le A descend en colonne 0, puis le B en colonne 10.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ le A finit en (0, 5) (0,5)

✓ le B finit en (10, 5) (10,5)

Ensuite : **0.22. Remonter : deplace_y et un pas négatif** — niveau 0 sur 10. · gameboy3 — écrire une cartouche Game Boy en C++